ExceeLearn — Project Documentation

Team: ExceeLearn
Track: Microsoft AI Track
Date: March 2026
Version: 2.0.0

Demo Video: https://youtu.be/zDf56Db8Hq0

Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. Solution Overview
4. Architecture
5. AI Methodology
6. Feature Implementation
7. Backend API Reference
8. Frontend Design
9. Testing & Observations
10. Responsible AI
11. Limitations & Trade-offs
12. Future Work

---

1. Executive Summary

ExceeLearn is an AI-powered student learning companion that models each student's evolving knowledge state and provides personalized, explainable, and actionable guidance to improve learning outcomes.

At its core, ExceeLearn implements the **Free Spaced Repetition Scheduler (FSRS)** cognitive memory model to track per-chapter knowledge stability, difficulty, and retrievability across five university courses. This is augmented by a **Retrieval-Augmented Generation (RAG)** pipeline that grounds AI responses in the student's own uploaded lecture materials, delivering contextually accurate answers with source citations.

The platform surfaces four distinct AI agents — Analyser, Planner, and Mentor — each tuned for a specific educational task, all communicating through a persistent, non-intrusive docked interface. Every AI-generated insight includes a **"Why?" explainability mechanism** that shows the student exactly which data drove the recommendation.

Key capabilities implemented and verified:

- FSRS memory model (Stability, Difficulty, Retrievability) with per-quiz real-time updates
- Hybrid vector + keyword RAG using Azure AI Search (3072-dim Gemini embeddings) + Google Gemini 2.5 Flash Lite
- Three specialized AI agents with distinct system prompts and temperature settings
- 16-week activity heatmap with inactivity gap detection
- Smart study timer with cognitive fatigue thresholds and session-based FSRS updates
- AI-generated study plans prioritizing critical chapters by stability rank
- Gamified achievement system and visual gap radar charts
- Firebase Hosting (frontend) + Docker/Cloud Run (backend)

---

2. Problem Statement & Motivation

Modern students interact with digital learning platforms daily — submitting quizzes, reviewing lecture PDFs, attempting practice problems — yet the data generated by these interactions is rarely translated into personalized, actionable insight. Common questions go unanswered:

- *What concepts am I genuinely weak at versus making careless mistakes?*
- *Am I improving, stagnating, or regressing over time?*
- *What should I focus on if I only have 3 days before my exam?*
- *Why do I keep failing the same type of question?*

The deeper challenge is that **learning is non-linear**. Students experience inactivity gaps, intensive revision bursts, and varying engagement across topics. A fixed rubric cannot adapt to these patterns. Memory itself decays — a topic understood last month may be forgotten today. Standard platforms ignore this temporal dimension entirely.

ExceeLearn addresses this by:

1. **Modelling memory scientifically** — using FSRS to track how long knowledge persists and when it needs reinforcement
2. **Grounding AI in lecture materials** — ensuring recommendations relate to actual course content, not generic advice
3. **Making AI decisions transparent** — every insight shows its data source, preserving student trust
4. **Adapting to behaviour over time** — stability scores evolve with every quiz attempt and study session

---

3. Solution Overview

ExceeLearn is a full-stack web application structured around three pillars:

Pillar 1 — Memory Modelling (FSRS)

Every chapter in every subject has a live memory profile comprising three components:

- **S (Stability):** Days before retrievability falls below 90% — a direct measure of how deeply the material is encoded
- **D (Difficulty):** Inherent hardness of the material [1–10], updated after each quiz
- **R (Retrievability):** Current probability of successful recall, computed as `R = 0.9^(t/S)`

These values update after every quiz and study session, producing a continuously evolving knowledge state.

Pillar 2 — RAG-Grounded AI Agents

Three AI agents (Analyser, Planner, Mentor) share a common RAG pipeline:

1. Student uploads lecture PDFs → chunked, embedded with `gemini-embedding-001` (3072-dim), indexed in Azure AI Search
2. Agent receives a query → hybrid vector+keyword search retrieves top-3 relevant chunks
3. Chunks injected into Gemini 2.5 Flash Lite system prompt → grounded response with inline source citations

Each agent has a distinct temperature and system prompt optimized for its task.

Pillar 3 — Explainable Insights Dashboard

The dashboard synthesizes FSRS state and AI outputs into:

- Global mastery donut chart (weighted across all subjects)

- Per-chapter stability bars with status badges (Mastered/Good/Review/Critical)
- "Why?" tooltips on every recommendation showing S, D, R values and the data source
- 16-week activity heatmap with inactivity gap alerts
- Gap radar chart visualizing weak vs strong chapter axes
- Gamified achievements panel

---

4. Architecture

System Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                         STUDENT BROWSER                          │
│  React 19 + Vite + TailwindCSS → Firebase Hosting               │
│                                                                  │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────────┐  │
│  │LoginView │ │ AppLayout  │ │SubjectView│ │   AgentDock    │  │
│  │          │ │Dashboard   │ │FSRS Cards │ │Analyse/Plan/   │  │
│  │Email auth│ │Right Rail  │ │Gap Radar  │ │Mentor (RAG)    │  │
│  └──────────┘ └──────────┘ └──────────┘ └──────────────┘  │
└──────────────────────┬───────────────────────────────────────────┘
                       │ HTTP (X-User-ID header)
                       ▼
┌──────────────────────────────────────────────────────────────────┐
│              FASTAPI BACKEND (Docker / Cloud Run)                │
│                                                                  │
│  /api/chat          /api/quiz-result    /api/stability           │
│  /api/upload        /api/documents      /api/stability/session   │
│                                                                  │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────────────┐ │
│  │ stability.py │ │  gemini.py   │ │    azure_search.py      │ │
│  │ FSRS Engine  │ │ LLM + Embed  │ │  Hybrid RAG Retrieval   │ │
│  └──────────────┘ └──────────────┘ └──────────────────────┘ │
│  ┌──────────────┐ ┌──────────────┐                            │
│  │blob_storage  │ │firebase_auth │                            │
│  │PDF Storage   │ │Header Auth   │                            │
│  └──────────────┘ └──────────────┘                            │
└──────────┬──────────────────┬──────────────────┬──────────────┘
           │                  │                  │
           ▼                  ▼                  ▼
┌──────────────┐  ┌──────────────────┐  ┌──────────────────┐
│  Firestore   │  │  Azure AI Search │  │  Azure Blob      │
│  FSRS State  │  │  Vector Index    │  │  Storage (PDFs)  │
│  (per user)  │  │  (RAG chunks)    │  │                  │
└──────────────┘  └──────────────────┘  └──────────────────┘
                          │
                          ▼
                  ┌──────────────────┐
                  │  Google Gemini   │
                  │  2.5 Flash Lite  │
```

```
                  │  + Embedding 001 │
                  └──────────────────┘
```

Technology Stack

Layer	Technology	Justification
Frontend	React 19, Vite 7, TailwindCSS 3	Fast HMR development, utility-first styling
Backend	FastAPI 0.115, Python 3.11, Uvicorn	Async endpoints, automatic OpenAPI docs
LLM	Google Gemini 2.5 Flash Lite	Low latency, cost-effective for real-time responses
Embeddings	gemini-embedding-001 (3072-dim)	State-of-the-art semantic retrieval
Vector Search	Azure AI Search (HNSW, cosine)	Managed hybrid search, enterprise-grade
PDF Storage	Azure Blob Storage	Durable, user-scoped PDF persistence
State Storage	Google Firestore + JSON fallback	Serverless, per-user FSRS state
Deployment	Firebase Hosting + Docker/Cloud Run	Serverless frontend + containerised backend

---

5. AI Methodology

5.1 FSRS — Free Spaced Repetition Scheduler

ExceeLearn implements a simplified but scientifically grounded variant of the FSRS algorithm, a modern successor to the SM-2 algorithm used by Anki.

Core Formula

Retrievability decays exponentially with time:

```
R(t) = 0.9 ^ (t / S)
```

Where:

- `R(t)` = probability of recall at time `t` (days since last review)
- `S` = stability (days until R = 0.9)
- At `t = 0`: R = 1.0 (just reviewed)
- At `t = S`: R = 0.9 (review threshold)

Stability Update Rules

After each quiz, the grade is derived from the score:

Score Range	Grade	Meaning
≥ 90%	perfect	Strong recall
≥ 75%	good	Solid understanding
≥ 60%	hard	Partial recall
< 60%	fail	Memory gap

Stability then updates as:

```
S_perfect = S × 2.0 × (11 - D) / 10
S_good    = S × 1.5 × (11 - D) / 10
S_hard    = S × 0.8 × (11 - D) / 10
S_fail    = max(1.0, S × 0.5 × R)
```

The difficulty factor `(11 - D) / 10` ensures that harder material gains stability more slowly. On fail, a partial credit formula using current retrievability `R` prevents complete reset.

All values are clamped: `S ∈ [1, 365]`, `D ∈ [1, 10]`.

Difficulty Update Rules

```
fail:    D = min(10, D + 1.0)
hard:    D = min(10, D + 0.5)
good:    D unchanged
perfect: D = max(1,  D - 0.5)
```

Dashboard Score Computation

The stability bar and percentage displayed to students:

```
score = R × 60 + min(S / 30, 1.0) × 40
```

This blends current recall probability (60% weight) with long-term stability (40% weight), giving a holistic single metric that is intuitive to students.

Status Classification

Score	Badge	Colour
≥ 75	Mastered	Emerald
≥ 60	Good	Sky Blue
≥ 45	Review	Amber
< 45	Critical	Red

Cold-Start Seeding (No Quiz History Yet)

When a student has uploaded a PDF but not yet taken a quiz, ExceeLearn seeds initial S and D values from keyword analysis of the PDF content:

```
hard_ratio = hard_keyword_hits / (hard_hits + core_hits)
D_seed = clamp(1.0 + hard_ratio × 9.0, 1, 10)
S_seed = clamp(20.0 - hard_ratio × 15.0, 1, 20)
```

Chapter profiles contain curated lists of `core_keywords` (foundational concepts) and `hard_keywords` (advanced/complex concepts). A document heavy in hard keywords gets a higher initial difficulty and lower stability, directing the student's attention immediately.

Session-Based Updates

The Smart Timer triggers a session update on stop. Duration multipliers:

Duration	Multiplier	Rationale
< 15 min	No update	Insufficient encoding time
15–44 min	1.0×	Standard session
45–90 min	1.2×	Optimal focus zone (bonus)
90–120 min	1.0×	Fatigue onset
> 120 min	0.8×	Saturation penalty

Focus quality (derived from pause frequency) further modulates the grade assigned to the session:

```
focusQuality = max(0, 1.0 - pauseRate × 0.5)
≥ 0.75 → grade "good"
≥ 0.50 → grade "hard"
<  0.50 → grade "fail"
```

5.2 RAG Pipeline — Retrieval-Augmented Generation

Why RAG?

Generic LLM responses can hallucinate, contradict course-specific content, or reference material not taught in the student's programme. RAG grounds every AI answer in the student's own uploaded lecture notes, ensuring responses are:

- **Accurate** — derived from verified source material
- **Attributable** — every claim cites a specific PDF and page number
- **Personalised** — each student's index contains only their own documents

Pipeline Detail

Ingestion Phase (on PDF upload):

1. PDF uploaded → stored in Azure Blob Storage at path `{user_id}/{subject}/{filename}`
2. Text extracted with `pypdf` (page by page)
3. Chunked at ~500 words with 50-word overlap (sentence-boundary aware)
4. Each chunk embedded via `gemini-embedding-001` with task type `retrieval_document` → 3072-dimensional vector
5. Chunk stored in Azure AI Search with fields: `id`, `content`, `title`, `source`, `page`, `user_id`, `content_vector`

Retrieval Phase (on chat query):

1. Query embedded via `gemini-embedding-001` with task type `retrieval_query`
2. Hybrid search: vector k-NN (HNSW cosine) + BM25 keyword simultaneously
3. Filtered by `user_id` (strict per-student isolation)
4. Top-3 chunks returned, ranked by hybrid score
5. Fallback to keyword-only if vector query fails

Generation Phase:

1. Retrieved chunks injected into system prompt:

```
[Source: {title}, Page {page}]
{content}
```

2. Agent-specific system prompt applied (see below)
3. Gemini 2.5 Flash Lite generates response with inline `[Source, Page X]` citations
4. Response + `sources` array returned to frontend
5. Frontend renders source citation pills below the response

Azure AI Search Index Schema

Field	Type	Indexed
id	Edm.String (key)	Yes
content	Edm.String	Searchable (BM25)
title	Edm.String	Filterable
source	Edm.String	Filterable
page	Edm.Int32	Filterable
user_id	Edm.String	Filterable
content_vector	Collection(Edm.Single) 3072-dim	HNSW cosine

5.3 AI Agent Design

Three agents share the RAG pipeline but differ in system prompt, temperature, and output structure.

Agent 1 — Mentor (Concept Explanation)

- **Temperature:** 0.3 (precise, grounded)
- **Max tokens:** 1200
- **Persona:** Warm, encouraging senior student
- **Behaviour:** Explains concepts step by step, cites sources inline as `[Source, Page X]`, flags when material is not covered in uploaded PDFs
- **Use case:** "Explain Dijkstra's algorithm and its time complexity"

Agent 2 — Analyser (Gap Diagnosis)

- **Temperature:** 0.45 (slightly exploratory)
- **Max tokens:** 900
- **Persona:** Direct, clinical diagnostician
- **Behaviour:** Identifies WHY the student is struggling; returns Gap Type, Weak Areas, Root Cause, What to Focus On
- **Use case:** "Why do I keep failing Dynamic Programming questions?"

Agent 3 — Planner (Study Scheduling)

- **Temperature:** 0.2 (deterministic)
- **Max tokens:** 800
- **Persona:** Efficient, structured scheduler
- **Behaviour:** Produces tight study schedules with Priority Topics, per-session time breakdown, Total Estimated Time, Success Criteria
- **Use case:** "Create a 5-day study plan for my SC2001 exam"

Why These Temperature Settings?

The temperature choices reflect a deliberate trade-off between creativity and reliability:

- **Mentor** at 0.3: explanations must be accurate; slight variance allows natural phrasing
- **Analyser** at 0.45: diagnosis benefits from slightly broader pattern matching across symptoms
- **Planner** at 0.2: schedules must be consistent and deterministic; hallucinating extra days or unrealistic hours is harmful

Maker-Checker Design (Planned)

The current architecture is single-agent. A planned enhancement involves a maker-checker loop where the Analyser generates a diagnosis and the Planner validates it against the FSRS state before presenting to the student. This is documented in Section 12 (Future Work).

5.4 Explainable AI — The "Why?" Mechanism

Every AI-generated insight in ExceeLearn includes a "Why?" button. Clicking it reveals:

- The **data source** (Quiz History / PDF Analysis / Baseline)
- The **raw FSRS values** (S in days, D/10, R%, review count)
- A **plain-language explanation** of what the values mean

Example tooltip content:

```
S = 6 days  D = 7.5/10  R = 61%  (4 reviews)
Last score: 55%  Last review: 12 days ago

"Stability low — knowledge decays quickly for this topic.
 High difficulty means it takes longer to consolidate.
 Source: Quiz History"
```

This satisfies the Microsoft Track's transparency requirement: students can always trace a recommendation back to a specific, verifiable data point.

---

6. Feature Implementation

6.1 Knowledge Dashboard (Home View)

The dashboard provides a real-time snapshot of the student's learning state across all five subjects.

Global Mastery Donut Chart A pure SVG donut chart (no external charting library) displays a weighted average stability score across all subjects. The arc fills with an animation on load, colour-coded green (≥75%), indigo (≥50%), or red (<50%). Per-subject bars below show individual breakdown.

Exam Countdown Cards Two upcoming exams are displayed with urgency styling (red for ≤5 days, amber for ≤14 days). Each card shows days remaining, venue, readiness percentage, and top revision topics.

Continue Learning Strip A horizontally scrollable strip shows the four most recently accessed chapters with progress bars, time-to-complete estimates, and direct "Continue" links. This drives re-engagement with partially studied material.

Stats Row Four metric cards: Courses Enrolled (5), Hours Learned (87), Achievements (12), Streak (9 days). These are currently static but designed to be hydrated from a future analytics endpoint.

Right Rail (xl screens)

- Global Mastery panel
- Activity Heatmap (compact, 16-week)
- Achievements Panel (compact, badge grid)

6.2 Subject View — Chapter Memory Cards

When a student navigates to a subject (e.g. SC2001), they see three tabs:

Chapters Tab Six chapter cards displayed in a 2-column grid. Each card shows:

- Stability bar (animated, colour-coded per FSRS status)
- Status badge: Mastered / Good / Review / Critical
- Trend arrow (TrendingUp / TrendingDown / Minus)
- Quiz score, lecture count, last studied date
- Topic tags
- "Why?" tooltip with live FSRS data
- Direct "Quiz" button to launch practice

Cards are hydrated from `GET /api/stability/{subject}` on subject load, and **refresh automatically** after a quiz completes (via `refreshKey` state increment triggering a new fetch). This was a key bug fixed during development — previously the tab had to be navigated away and back to see updated scores.

Gap Radar Tab A radar/spider chart (pure SVG polygon, no Chart.js dependency) with one axis per chapter. Weak chapters (low stability) have shorter axes rendered in red; strong chapters have longer axes in indigo. Below the chart, a priority action list ranks chapters from lowest to highest stability with direct "Practice" links.

The radar uses live FSRS data whenever available from the backend. A bug was identified and fixed during development: the chart previously only used live data when `scoreAssessed` (PDF-derived) was true, ignoring quiz-derived scores. The fix passes live data whenever `liveScores` contains any entries.

Practice Tab A chapter selector panel (left) and QuizSimulator panel (right). The chapter selector shows stability and quiz score per chapter, with "critical" badges for chapters below 50%.

6.3 Quiz Simulator & FSRS Update

The quiz flow has three phases:

Idle Phase: "Start Practice Quiz" button with subject and chapter context.

Quiz Phase:

- 3 questions drawn from a static per-chapter question bank (2–5 questions per chapter, shuffled)
- Per-question timer (seconds)
- Progress bar (animated)
- Four option buttons with immediate correct/incorrect colour feedback
- Explanation text appears after selection (green = correct explanation, red = wrong + correct answer shown)
- "Next Question" / "See Results" navigation

Results Phase:

- Score circle (indigo ≥60%, red <60%)
- Grade label: Excellent / Good / Needs Work / Critical
- Stats: correct count, duration, average time per question
- Per-question review list with check/cross marks
- FSRS Memory Model panel:
    - Shows updated S (stability days), D (difficulty/10), Next Review date

- Stability bar with new score
- Grade trend icon (TrendingUp / Minus / TrendingDown)
- Memory consolidation message

After the results are shown, `onQuizComplete()` fires: `refreshKey` increments in SubjectView (triggering a fresh `GET /api/stability/{subject}` fetch) and the view switches back to the Chapters tab so the student immediately sees updated stability bars.

6.4 AI Agent Dock

The AgentDock is a persistent panel fixed to the bottom-centre of the screen (z-50, max-w-2xl). It is subject-context aware — when the student is viewing SC2001, all agents receive `subject: "SC2001"` in their API calls.

Collapsed State: Shows a single insight row (hardcoded per subject from `AGENT_CONTEXTS`) with a badge, a truncated detail, and the "Why?" button. A "Chat ↑" button expands the dock.

Expanded State: Full chat interface with message history, typing indicator, source citation pills, and RAG status banner.

RAG Status Banner (Mentor tab): fetches `GET /api/documents?subject={code}` on expansion to show "RAG active · N PDFs indexed for [subject]" or a warning if no PDFs are indexed.

Chat API call:

```
POST /api/chat
{ user_query, agent, subject }
→ { reply, agent, sources: [{title, source, page}], rag_used }
```

Source citations render as `📄 filename p.X` tags below each agent response. If `rag_used = false`, a `⚠ No lecture PDFs matched` warning is shown.

6.5 Study Plan Generator

Triggered by the "AI Study Plan" button on the subject header. The plan is generated client-side from the live FSRS chapter data using a deterministic algorithm:

1. Chapters sorted into three buckets: critical (stability < 45), review (45–60), good (60–75)
2. Available days = min(daysToExam, 7)
3. Day allocation:
   - Days 1–2: lowest-stability critical chapters (2h each)
   - Days 3–4: review chapters (1.5h each)
   - Days 5–6: good chapters (45 min, consolidation)
   - Final day: mock exam session (3h) if space permits
4. Each day has 3 auto-generated tasks: re-read lecture notes, practice problems, review quiz mistakes with AI Mentor

The modal displays a tab per day, colour-coded red (critical), amber (review), sky (good), with stability target indicators and task checklists.

Why client-side generation? Speed and determinism. The algorithm is simple enough to run in the browser with zero API latency, and uses the same FSRS data already loaded in state.

6.6 Smart Timer

The Smart Timer lives in the sidebar and tracks focused study sessions per subject.

States: idle → running → paused → done

Visual ring: An SVG circular progress ring that fills over 90 minutes. Colour changes:

- Indigo: 0–60 min (optimal zone)
- Amber: 60–80 min (approaching fatigue)
- Red: 80–90 min (fatigue threshold)

90-minute alert: When elapsed ≥ 90 min, a dismissible banner warns "90-minute cognitive limit reached — take a break."

Session end: On Stop, the timer calls `POST /api/stability/session` with `{ subject, duration_minutes, focus_quality, pause_count }`. The backend updates S and D for all chapters in that subject using the duration multiplier and focus quality grade. A summary modal shows duration, focus quality score, and retention bonus percentage.

Focus Quality Metric:

```
focusQuality = max(0, 1.0 - pauseRate × 0.5)
pauseRate = pauseCount / durationMinutes
```

A student who paused 3 times in a 20-minute session gets `pauseRate = 0.15`, `focusQuality = 0.925` (high).

6.7 Activity Heatmap

A 16-week grid (112 days) visualizing daily study activity. Each cell colour represents activity intensity:

- `bg-slate-800` : no activity
- `bg-indigo-900/60` : light
- `bg-indigo-700/70` : medium
- `bg-indigo-500` : high
- `bg-indigo-400` : intense

Inactivity Gap Detection: The heatmap identifies consecutive inactive weeks and renders a dashed gap indicator with an amber alert banner: "7-day inactivity gap detected — consider a review session."

Currently uses simulated data distributed across the 16-week window. The data structure is designed to be replaced by real session timestamps from the backend.

6.8 Achievements Panel

Twelve achievements across four categories: Study Habits, Quiz Performance, Consistency, and Mastery Milestones. Each achievement has:

- An icon and title
- Unlock condition (e.g., "Complete 5 quizzes with score ≥ 80%")
- A progress bar (current / target)
- Locked/unlocked visual state

The compact mode (right rail) shows a badge grid of unlocked achievements. The full-page view shows all achievements with progress tracking.

Currently achievements are evaluated against hardcoded demo data. The evaluation logic is implemented and ready to be wired to real backend analytics.

6.9 PDF Upload

The UploadModal allows students to upload lecture PDFs per subject. The flow:

1. File picker (PDF only, max 10MB validation on frontend)
2. Subject selector (defaults to current subject if viewing a subject page)
3. `POST /api/upload` (multipart form) → backend chunks, embeds, indexes
4. Success state shows: filename, subject, pages processed, chunks indexed

Uploaded PDFs are listed via `GET /api/documents` and can be deleted individually via `DELETE /api/documents/{blob_path}`.

---

7. Backend API Reference

All endpoints require `X-User-ID: {email}` header for user identity.

GET /api/health

Health check. Returns service status, model versions, index name.

```
{ "status": "ok", "version": "2.0.0", "index": "excelearn-index",
  "model": "gemini-2.5-flash-lite", "vector": "gemini-embedding-001 (3072 dims)" }
```

POST /api/chat

RAG-grounded agent response.

Request:

```
{ "user_query": "Explain Dijkstra's algorithm",
  "agent": "mentor",
  "subject": "SC2001" }
```

Response:

```
{ "reply": "Dijkstra's algorithm is... [Source: SC2001_Lecture5, Page 12]",
  "agent": "mentor",
  "sources": [{ "title": "SC2001_Lecture5", "source": "lecture5.pdf", "page": 12 }],
  "rag_used": true }
```

POST /api/quiz-result

Submit quiz score and receive updated FSRS state.

Request:

```
{ "subject": "SC2001", "chapter_id": "ch04",
  "score": 55.0, "total_questions": 3, "correct": 1 }
```

Response:

```
{ "subject": "SC2001", "chapter_id": "ch04", "grade": "fail",
  "S": 3.2, "D": 8.5, "R": 0.61, "stability": 47,
```

```
  "status": "critical", "bar_color": "#EF4444",
  "reviews": 5, "last_review": "2026-03-03",
  "last_score": 55.0, "next_review_in": 3 }
```

GET /api/stability/{subject}

All chapter FSRS scores for a subject.

Response:

```
{ "subject": "SC2001", "assessed": true,
  "chapters": [
    { "chapter_id": "ch01", "title": "Algorithm Basics",
      "stability": 72, "status": "good", "bar_color": "#0ea5e9",
      "S": 18.0, "D": 3.5, "R": 0.88, "reviews": 5,
      "last_review": "2026-02-28", "last_score": 88.0,
      "next_review_in": 12, "assessed": true }
  ] }
```

GET /api/stability/{subject}/{chapter_id}

Detailed FSRS state + explainability data for one chapter.

Response includes: all stability fields + `data_source` (quiz_history | pdf_keywords | baseline) + `why_explanation` (plain-language string for "Why?" tooltip).

POST /api/stability/session

Record study session and update all chapters in subject.

Request:

```
{ "subject": "SC2001", "duration_minutes": 47.5,
  "focus_quality": 0.85, "pause_count": 1 }
```

Response: Updated S/D/status for all 6 chapters in the subject.

POST /api/upload

Upload and index a PDF. Multipart form: `file` (PDF), `subject` (string).

Response:

```
{ "success": true, "filename": "SC2001_Lecture5.pdf",
  "subject": "SC2001", "blob_path": "user@email.com/SC2001/SC2001_Lecture5.pdf",
  "pages_processed": 18, "chunks_indexed": 42 }
```

GET /api/documents

List indexed PDFs for the user, optionally filtered by subject.

DELETE /api/documents/{blob_path}

Remove a PDF from Blob Storage and delete all its indexed chunks from Azure Search.

---

8. Frontend Design

8.1 Design Language

ExceeLearn uses a dark-first design system built on TailwindCSS 3 with a custom colour palette:

- **Background:** `#020617` (near-black slate)
- **Surface:** `slate-900/80` with `backdrop-blur` (glassmorphism)
- **Primary accent:** Indigo (600/500/400) — interactive elements, active states
- **Status colours:** Red (critical), Amber (review/warning), Sky (good), Emerald (mastered/success)
- **Typography:** Inter (body), Geist Mono (data, badges, code)

The aesthetic is inspired by professional data dashboards — dense information presentation without visual clutter. All animations are CSS-based (no external animation libraries): opacity fades, translateY slides, stroke-dasharray arcs, and keyframe breath-ring pulses.

8.2 Component Architecture

All components are function-based React (no class components). State management is:

- **Local state** (`useState`): UI-only state (active tab, modal open/closed)
- **Context** (`useContext`): Auth state (`AuthContext`) and timer state (`TimerContext`)
- **Derived state**: FSRS data merged from static `subjectData.js` + live API response in `SubjectView`

No external state management library (Redux, Zustand, Jotai) — the app is simple enough that context + prop drilling covers all cases cleanly.

8.3 Performance Decisions

- **No charting library:** Both the Donut chart and Gap Radar are pure SVG, saving ~50KB of bundle weight
- **Lazy stability fetch:** FSRS data is fetched once per subject navigation, not on every render
- **refreshKey pattern:** Re-fetches are triggered by incrementing a counter, avoiding complex cache invalidation
- **Vite with code splitting:** Production build uses Rollup for optimal chunk sizes
- **Firebase CDN:** Static assets served from Firebase global CDN with 1-year cache headers for JS/CSS/fonts

8.4 Responsiveness

- **Sidebar:** Hidden on mobile (hamburger menu), shown on `lg:` screens
- **Right rail:** Only shown on `xl:` screens (≥1280px)
- **Chapter grid:** 1-column on mobile, 2-column on `md:` screens
- **Agent Dock:** Full-width on mobile, max-w-2xl centered on desktop

---

9. Testing & Observations

9.1 Test User Setup

A dedicated test account `test@test.com` has a pre-seeded FSRS state (`fsrs_state_test@test.com.json`) representing approximately 6 weeks of realistic study behaviour across all 5 subjects and 30 chapters.

Key characteristics of the test state:

- **SC1007/ch06** (Graph Algorithms): S=3.5, D=8.5, score=41% — extreme critical case

- **SC2001/ch04** (Dynamic Programming): S=6.0, D=7.5, score=55% — critical
- **SC2001/ch06** (NP-Completeness): S=4.5, D=8.0, score=38% — extreme critical
- **SC2002/ch01** (OOP Principles): S=28.0, D=2.5, score=93% — mastered case
- Mix of recent reviews (≤3 days) and stale reviews (>21 days) to demonstrate decay

9.2 FSRS Verification Tests

Test 1 — Score Drop (fail grade)

- Initial state: SC2001/ch04, S=6.0, D=7.5
- Quiz score: 33% (1/3 correct) → grade "fail"
- Expected new S = max(1.0, 6.0 × 0.5 × R) where R = 0.9^(t/6)
- Observed: S drops to ~3.2 days, D increments to 8.5
- Result: ✅ Matches formula

Test 2 — Score improvement (good grade)

- Initial state: MH1810/ch05, S=3.0, D=8.0
- Quiz score: 80% → grade "good"
- Expected new S = 3.0 × 1.5 × (11-8.0)/10 = 3.0 × 1.5 × 0.3 = 1.35... → clamped to 1.35
- Observed: S updates to ~1.4 days (difficulty factor limits growth for hard material)
- Result: ✅ Matches formula. Hard material resists rapid stability gain — by design.

Test 3 — Perfect score (mastered chapter)

- Initial state: SC2002/ch01, S=28.0, D=2.5
- Quiz score: 100% → grade "perfect"
- Expected new S = 28.0 × 2.0 × (11-2.5)/10 = 28.0 × 2.0 × 0.85 = 47.6
- Observed: S ≈ 47.6 days
- Result: ✅ Easy material gains stability quickly

Test 4 — Tab refresh after quiz

- Issue: Stability bars on Chapters tab did not update after quiz without navigating away
- Root cause: `useEffect` only re-fetched on `subjectCode` change; no re-fetch on quiz completion
- Fix: Added `refreshKey` state; `onQuizComplete` callback increments `refreshKey`, triggering re-fetch
- Result: ✅ Bars update immediately on returning to Chapters tab

Test 5 — Gap Radar with quiz data

- Issue: Radar chart showed static data even after multiple quizzes
- Root cause: `liveChapters` prop only passed when `scoreAssessed=true` (PDF keyword seeding), ignoring quiz-derived scores
- Fix: Changed condition to `liveScores && Object.keys(liveScores).length > 0`
- Result: ✅ Radar now reflects all quiz-derived FSRS data

Test 6 — RAG with SC2001 lecture PDF

- Uploaded SC2001 Graph Algorithms lecture HTML as PDF equivalent
- Query: "Explain Dijkstra's algorithm and its time complexity"
- Retrieved 3 chunks from lecture notes (pages 4, 7, 12)
- Response correctly cited `[Source: SC1007_Graph_Algorithms, Page 7]`
- `rag_used: true` confirmed in response
- Result: ✅ Full RAG pipeline functional end-to-end

9.3 UI/UX Observations

- **Why? button visibility:** Initially `text-[10px] text-slate-600` — near-invisible on dark background. Upgraded to `text-xs font-semibold text-indigo-400` with a pill background (`bg-indigo-500/15 border border-indigo-500/40`) for clear discoverability.
- **Login text:** "Enter Dashboard" updated to "Sign In" for clarity. Subtitle updated to match product positioning.
- **AchievementsPanel in right rail:** Was missing from home dashboard right rail. Added alongside `ActivityHeatmap` so the wrap-up dashboard view shows all three right-rail panels.
- **Agent Dock context:** Correctly propagates subject code when viewing a subject; defaults to "Global" on home view.

9.4 Known Behaviours

- **Question bank is static:** Questions are hardcoded per chapter in the frontend. Each chapter has 2–5 questions; 3 are randomly drawn per quiz. This is intentional for the prototype — a production system would generate questions dynamically via the Mentor agent.
- **Stats row is static:** Hours Learned (87), Streak (9 days), Achievements (12) are hardcoded. A future `/api/analytics` endpoint would hydrate these.
- **Activity Heatmap is simulated:** The 16-week heatmap uses procedurally generated data. A production system would log session timestamps to Firestore and aggregate.
- **Auth is email-only:** No password verification. `X-User-ID` header passes email directly. Suitable for a hackathon prototype; production would use Firebase ID tokens.

---

10. Responsible AI

10.1 Explainability

Every AI recommendation in ExceeLearn is traceable to a specific data source:

Recommendation	Data Source	Shown to Student
"Chapter is Critical"	FSRS: S=3.2d, D=8.5, R=61%	"Why?" tooltip with S/D/R values
"Focus on DP first"	Lowest stability rank in study plan	Day label + stability % shown
Mentor answer	PDF chunks cited inline	📄 source pills with page numbers
Analyse insight	Quiz history + FSRS state	"Why?" tooltip: "Source: Quiz History"

No recommendation is ever shown without a traceable data source. This is enforced at the component level — every `WhyTooltip` requires a `source` prop; every agent response includes `sources` and `rag_used` .

10.2 Consistency & Determinism

- **FSRS updates are deterministic:** Same score on same chapter always produces the same S/D delta. No randomness in the memory model.
- **Study plans are deterministic:** Given the same FSRS state and exam date, the plan algorithm always produces the same output. Client-side generation ensures no LLM variability in scheduling.
- **Agent temperatures:** Deliberately set low (0.2–0.45) to maximise response consistency. The Planner at 0.2 is near-deterministic.

10.3 Fairness & Bias Considerations

- **No demographic data used:** FSRS operates purely on quiz scores and time elapsed. No student attributes (gender, background, prior education) influence recommendations.

- **Difficulty is content-based:** The `D` parameter reflects the inherent complexity of material keywords, not the student's identity.
- **Equal treatment:** All chapters start at the same baseline (S=7, D=5) before any quiz history, regardless of which student uses the system.

10.4 Privacy & Data Minimisation

- **Minimal data collection:** Only email (used as user ID) and quiz scores are stored. No behavioural tracking beyond session duration.
- **Per-user data isolation:** Azure Search filters by `user_id` on every query. One student's PDFs cannot appear in another student's RAG results.
- **No data sharing:** No third-party analytics or advertising SDKs. Firebase Hosting serves static files only.
- **Local fallback:** FSRS state falls back to local JSON files if Firestore is unavailable — no data is lost.

10.5 Human Agency

- **Students control the plan:** The study plan is a suggestion, not a directive. The modal has "Export" and "Accept" buttons but no enforcement mechanism.
- **Quiz is optional:** Students can view stability scores without taking quizzes. The system never forces interaction.
- **Override mechanism:** Students can re-take quizzes immediately. A poor quiz result that incorrectly tanks a score can be corrected by re-taking.
- **Why? is always accessible:** Every recommendation has an accessible explanation. Students are never told "trust us" — the data is always shown.

---

11. Limitations & Trade-offs

11.1 Acknowledged Limitations

FSRS Simplification The full FSRS-5 algorithm uses a 17-parameter model calibrated on millions of Anki flashcard reviews. Our implementation uses a simplified 3-parameter variant (S, D, R) with hand-tuned weights. The core decay formula `R = 0.9^(t/S)` and the grade-based update rules are accurate, but the exact weight values (2.0, 1.5, 0.8, 0.5) are not calibrated to educational assessment data — they are informed approximations.

Static Question Bank Quiz questions are hardcoded frontend data (2–5 questions per chapter, 30 chapters = ~90 total questions). This is sufficient for demonstrating the FSRS update mechanism but would not scale to a real educational platform. Production would require dynamic question generation via the AI Mentor agent or integration with a question bank API.

Single-Session Auth Authentication is email-only with no password, JWT, or session expiry. This is a deliberate simplification for the hackathon prototype. All claims about user data isolation refer to the backend's `X-User-ID` filtering, which trusts the client-supplied email.

Subject Coverage Five subjects are hardcoded with six chapters each. The system is architected for extensibility (any `subject` string works in the API), but the frontend subject switcher and chapter data are static. Adding a new subject requires editing `subjectData.js`.

Activity Heatmap The 16-week heatmap uses procedurally generated data. It visually demonstrates the inactivity gap detection feature but does not reflect real session history. Real session timestamps are logged to Firestore via `POST /api/stability/session` but the heatmap does not yet query this data.

11.2 Trade-offs

Decision	Chosen Approach	Alternative	Reason

Study plan generation	Client-side algorithm	LLM-generated plan	Speed, determinism, no API cost
Charts	Pure SVG	Chart.js / Recharts	Bundle size, custom animation control
Auth	Email header	Firebase ID tokens	Prototype simplicity
FSRS storage	Firestore + JSON fallback	SQL database	Serverless, per-user document model
State management	React Context	Redux/Zustand	App complexity doesn't warrant library overhead
Embeddings	gemini-embedding-001 (3072-dim)	text-embedding-3-small	Best semantic quality for RAG
LLM	Gemini 2.5 Flash Lite	GPT-4o mini	Lower latency for real-time chat

---

12. Future Work

12.1 Maker-Checker Agent Loop

The current architecture is single-agent (one RAG call per query). A planned enhancement:

1. **Analyser agent** diagnoses gaps from FSRS data
2. **Validator agent** checks the diagnosis against quiz history for consistency
3. If inconsistency detected (e.g., Analyser says "weak at recursion" but recent quiz score was 90%), flag for student review
4. Student can accept or reject the diagnosis (human-in-the-loop)

12.2 Dynamic Question Generation

Replace the static question bank with Mentor-agent-generated questions grounded in uploaded PDFs:

- `POST /api/generate-questions { subject, chapter_id, count, difficulty }`
- Questions generated fresh each quiz, ensuring variety and relevance to course content
- Adaptive difficulty: harder questions for high-stability chapters, easier for critical chapters

12.3 Real Activity Tracking

Connect `ActivityHeatmap` to real Firestore session logs:

- `POST /api/stability/session` already writes to Firestore
- Add `GET /api/sessions?weeks=16` endpoint aggregating session timestamps
- Heatmap queries this endpoint and renders real activity data

12.4 Multi-Platform Integration

- **NTULearn/LMS import:** Parse quiz results from Blackboard/Canvas export files to seed FSRS history without manual quizzes
- **Calendar integration:** Export study plan to Google Calendar / iCal format
- **Push notifications:** Remind students when a chapter's R drops below 0.7

12.5 Calibrated FSRS Parameters

Apply Bayesian optimisation over collected quiz data to tune the weight parameters (currently hand-set at 2.0, 1.5, 0.8, 0.5) to match real learning curves in university assessment contexts.

12.6 Collaborative Features

- **Peer comparison:** Anonymised stability benchmarks showing where a student ranks within their cohort on each chapter
- **Group study plans:** Merge FSRS states across study group members to identify shared gaps

---

Appendix A — FSRS State Schema

```
{
  "SC2001/ch04": {
    "S": 6.0,
    "D": 7.5,
    "last_review": "2026-02-20",
    "reviews": 4,
    "last_score": 55.0
  }
}
```

Fields:

- `S` (float): Stability in days. R = 0.9^(days_since_review / S)
- `D` (float, 1–10): Difficulty. Higher = slower stability growth
- `last_review` (ISO date or null): Date of last quiz submission
- `reviews` (int): Cumulative review count
- `last_score` (float, 0–100 or null): Most recent quiz percentage

Appendix B — Subject & Chapter Reference

Subject	Code	Chapters	Exam Date
Data Structures & Algorithms	SC1007	6	4 Mar 2026
Linear Algebra	MH1810	6	11 Mar 2026
Algorithm Design & Analysis	SC2001	6	24 Mar 2026
Object-Oriented Design	SC2002	6	31 Mar 2026
Computer Organisation	SC2005	6	7 Apr 2026

Appendix C — Environment Configuration

Backend (.env):

```
EXTERNAL_API_KEY=           # Google Gemini API key
AZURE_SEARCH_ENDPOINT=      # Azure AI Search endpoint URL
AZURE_SEARCH_KEY=           # Azure AI Search admin key
```

```
AZURE_SEARCH_INDEX_NAME=    # excelearn-index
AZURE_BLOB_CONNECTION_STRING= # Azure Storage connection string
```

Frontend (.env.local):

```
VITE_API_BASE=http://localhost:8000   # Backend URL
```

Deployment:

- Frontend: `npm run build` → `firebase deploy --only hosting`
- Backend: `docker build -t excelearn-backend . && docker run -p 8000:8080 excelearn-backend`
- Cloud Run: Container deployed to Google Cloud Run with PORT=8080

References

[1] J. Ye, "FSRS: A Modern Spaced Repetition Algorithm," *GitHub*, 2022. [Online]. Available: https://github.com/open-spaced-repetition/fsrs4anki

[2] P. A. Wozniak, "SM-2 Algorithm," *SuperMemo*, 1990. [Online]. Available: https://www.supermemo.com/en/blog/application-of-a-computer-to-improve-the-results-obtained-in-working-with-the-supermemo-method

[3] Google, "Gemini API Documentation," *Google AI for Developers*, 2024. [Online]. Available: https://ai.google.dev/docs

[4] Google, "gemini-embedding-001 Text Embedding Model," *Google AI*, 2024. [Online]. Available: https://ai.google.dev/gemini-api/docs/embeddings

[5] Microsoft, "Azure AI Search Documentation," *Microsoft Learn*, 2024. [Online]. Available: https://learn.microsoft.com/azure/search/

[6] Microsoft, "Retrieval-Augmented Generation (RAG) with Azure AI Search," *Microsoft Learn*, 2024. [Online]. Available: https://learn.microsoft.com/azure/search/retrieval-augmented-generation-overview

[7] Microsoft, "Azure Blob Storage Documentation," *Microsoft Learn*, 2024. [Online]. Available: https://learn.microsoft.com/azure/storage/blobs/

[8] Google, "Firebase Hosting Documentation," *Firebase*, 2024. [Online]. Available: https://firebase.google.com/docs/hosting

[9] Google, "Cloud Firestore Documentation," *Firebase*, 2024. [Online]. Available: https://firebase.google.com/docs/firestore

[10] Meta, "React — A JavaScript Library for Building User Interfaces," *React*, v19.2, 2024. [Online]. Available: https://react.dev/

[11] E. Osmani, "Vite — Next Generation Frontend Tooling," *vitejs.dev*, v7.3, 2024. [Online]. Available: https://vite.dev/

[12] A. Wathan, "Tailwind CSS — A Utility-First CSS Framework," *Tailwind Labs*, v3.4, 2024. [Online]. Available: https://tailwindcss.com/

[13] S. Ramirez, "FastAPI — Modern, Fast Web Framework for Building APIs with Python," *GitHub*, v0.115, 2024. [Online]. Available: https://fastapi.tiangolo.com/

[14] Radix UI, "Radix Primitives — Unstyled, Accessible UI Components for React," 2024. [Online]. Available: https://www.radix-ui.com/

[15] J. Rutherford, "Lucide Icons," *GitHub*, 2024. [Online]. Available: https://lucide.dev/

[16] pypdf Contributors, "pypdf — A Pure Python PDF Library," *GitHub*, 2024. [Online]. Available: https://github.com/py-pdf/pypdf

[17] H. Ebbinghaus, "Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie," *Duncker & Humblot*, Leipzig, 1885. *(Foundational memory decay theory underlying FSRS)*

ExceeLearn — AI-powered student learning companion
Built for the Microsoft AI Track, NTU Hack&Roll / Hackathon 2026